4.9. Python и ИБ форензика и разбор событий

4. Python в задачах информационной безопасности

В этом уроке

- Узнаем что такое форензика (цифровая криминалистика)
- Повторим основы анализа логов
- Научимся выявлять следы атак
- Поймем как строить хронологии событий

Цифровая форензика

Цифровая форензика — это область информационной безопасности, которая занимается сбором, анализом, интерпретацией и представлением цифровых доказательств для расследования инцидентов.

Цель форензики - восстановить:

- Кто — пользователь или злоумышленник
- Что — было сделано (удаление файла, установка трояна)
- Когда — дата и время действий
- Где — локально или по сети, IP-адрес, рабочая станция
- Как — с помощью какой команды, уязвимости, инструмента

Цифровая форензика

Когда применяется:

- После взлома системы
- При подозрении на утечку данных
- В рамках аудита безопасности
- При судебных разбирательствах

Источники информации для форензики

1. Системные журналы
    1. /var/log/auth.log, /var/log/syslog, /var/log/secure
    2. Показывают входы, sudo, ssh-доступ, ошибки авторизации
2. Журналы веб-серверов
    1. Apache/Nginx: access.log, error.log
    2. Можно отследить сканирование сайта, инъекции, брутфорс
3. Журналы Windows (EVTX)
    1. Содержат данные о входе в систему, запуске процессов, ошибках
4. Сетевые дампы (pcap)
    1. Перехваченный трафик: DNS-запросы, HTTP, FTP, SSH
5. История браузеров, временные файлы, кэш
6. Метаданные файлов
    1. Дата создания, последнего изменения, последний доступ

Роль Python в форензике

- Python — универсальный язык для автоматизации рутинных задач. Его роль в форензике:
- 🔧 Преимущества:
  - Быстрая обработка текстовых логов
  - Удобный парсинг, фильтрация, группировка
  - Встроенные библиотеки (re, datetime, os) позволяют без внешних зависимостей делать многое
- 📦 Популярные библиотеки:
  - re — регулярные выражения (поиск нужных строк)
  - datetime — работа с датами и временем
  - pandas — анализ больших объёмов логов
  - pyshark — анализ сетевых дампов .pcap
  - logparser — работа с Windows-журналами

Парсинг логов: общая структура

Что нужно извлечь:

- дата и время
- Служба
- тип события
- IP-адрес источника

Парсинг логов с помощью Python

```
import re

log = "Jun 20 14:55:10 server sshd[1234]: Failed password for invalid user admin from 192.168.1.1
pattern = r"(\w+ \d+ \d+:\d+:\d+).*from (\d+\.\d+\.\d+\.\d+)"
match = re.search(pattern, log)

if match:
    print("Дата:", match.group(1))
    print("IP-адрес:", match.group(2))
```

Регулярное выражение ищет:

- \w+ \d+ \d+:\d+:\d+ — дату (Jun 20 14:55:10)
- from <ip> — IP-адрес

Обработка файла логов

- Пример простейшего скрипта, фильтрующего неудачные входы:

```
with open("auth.log") as f:
    for line in f:
        if "Failed password" in line:
            print(line.strip())
```

- 📌 Можно дополнить:
  - Подсчётом частоты попыток входа
  - Поиском по диапазону дат
  - Выводом IP и времени
  - Сохранять в CSV
  - Исключать внутренние IP
  - Добавить ограничение по времени (например, только последние 24 часа)

Выявление следов атаки

Что искать в логах:

🔺 Множественные неудачные попытки входа

→ Признак брутфорса или подбора пароля.

🔺 Запуск подозрительных или нестандартных команд

→ wget, curl, python, bash, nmap, nc, perl — часто встречаются при атаках.

🔺 Доступ к критичным системным файлам

→ /etc/passwd, /etc/shadow, ~/.ssh/authorized_keys

🔺 Подключения с подозрительных или внешних IP-адресов

📌 Методика анализа:

Используем регулярные выражения (re) для выделения нужных фрагментов.

Считаем частоту появлений с помощью collections.Counter.

Сбор статистики об атаках

Цель: найти IP-адреса, с которых чаще всего предпринимались попытки входа в систему

```
import re
from collections import Counter

ips = []
with open("auth.log") as f:
    for line in f:
        if "Failed password" in line:
            ip = re.search(r"from (\d+\.\d+\.\d+\.\d+)", line)
            if ip:
                ips.append(ip.group(1))

print(Counter(ips).most_common(5))
```

Построение timeline (временной линии)

📌 Для чего нужен timeline:

Понимание последовательности действий атакующего

Выявление момента начала инцидента

Установка временных связей между событиями

📌 Что нужно:

Вытянуть из лога даты и события

Привести всё к единому формату времени

Отсортировать и, по возможности, визуализировать

Python: создание timeline

Пример простейшего скрипта timeline из auth.log:

```
from datetime import datetime

events = []
with open("auth.log") as f:
    for line in f:
        if "Failed password" in line:
            try:
                # Преобразуем дату: "Jun 20 14:55:10" → datetime
                date = datetime.strptime(line[:15], "%b %d %H:%M:%S")
                events.append((date, line.strip()))
            except ValueError:
                continue

events.sort()
for event in events:
    print(event[0], "->", event[1])
```

Восстановление последовательности событий

- Цель - понять, что делал злоумышленник после входа.
- 📌 Методика:
- Ищем временные связанные события:
  - Вход по SSH
  - Установка соединения или загрузка файла
  - Выполнение команд (rm, curl, scp, python, nano)
  - Попытка скрыть следы
- Как искать:
  - По ключевым словам
  - По времени — события с разницей в 1–2 минуты, идущие друг за другом

Автоматизация: скрипт анализа инцидента

🛠️ Что делает скрипт:

- Открывает лог
- Фильтрует строки по ключевым признакам атаки (например, Failed password, wget, scp, rm)
- Сохраняет результаты во временной линии
- Выделяет аномальную активность:
  - повторяющиеся IP
  - повторяющиеся пользователи
- Формирует отчёт (например, в CSV)

Автоматизация: скрипт анализа инцидента

```python
from collections import Counter
import re, csv

events = []
ips = []

with open("auth.log") as f:
    for line in f:
        if "Failed password" in line or "Accepted password" in line:
            events.append(line.strip())
            ip = re.search(r"from (\d+\.\d+\.\d+\.\d+)", line)
            if ip:
                ips.append(ip.group(1))

# Запись отчёта
with open("incident_report.csv", "w") as out:
    writer = csv.writer(out)
    writer.writerow(["Событие"])
    for e in events:
        writer.writerow([e])

print("Top IPs:", Counter(ips).most_common(5))
```

Меры защиты и рекомендации

🛡️ Технические меры:

- Запрет на выполнение скриптов в директориях загрузки (uploads)
- Отключение exec() и shell_exec() в PHP
- Установка правил доступа через .htaccess или nginx/apache config

Меры защиты и рекомендации

📋 Организационные меры:

- Регулярный аудит логов
- Настройка автоматического анализа подозрительных IP
- Использование средств наподобие Fail2Ban

🔄 Автоматизация:

- Сценарии автоанализа логов с Python
- Ежедневные отчёты по активности с фильтрацией по IP, пользовательским агентам и аномалиям

Итоги урока

- Что узнали что такое цифровая форензика
- Изучили какие логи анализируются при расследовании
- Поняли как Python помогает автоматизировать процесс анализа
- Разобрались как восстанавливать последовательность событий (timeline)Как выявлять следы атак и строить отчёты